

Flutter Web Object Detection

Real-time Object Detection in the Browser using TensorFlow.js coco-ssd

Project Report & Technical Documentation

Project Name	flutter_object_detection_web
Platform	Flutter Web (Browser only)
Model	TensorFlow.js coco-ssd (pretrained)
Key Packages	image_picker, js, dart:html / dart:js_util
Date	August 2026

1. Overview

This project demonstrates **real-time object detection** running entirely inside a web browser. A user picks an image from the gallery; the Flutter app draws labeled bounding boxes on top of the image and lists each detected class with its confidence score.

Rather than running a pure Dart/TFLite model, the solution bridges into JavaScript via Flutter’s web platform view and `dart:js_util`. The heavy lifting is performed by the pretrained **coco-ssd** model from TensorFlow.js, which is loaded from a CDN. This approach keeps the Flutter codebase lightweight while leveraging mature, well-optimized browser ML libraries.

2. Key Features

- **Gallery image pick** – Uses the official `image_picker` package (web-compatible).
- **Browser-native inference** – coco-ssd runs via TensorFlow.js; no native plugins or server round-trips required.
- **Accurate bounding-box overlay** – Correctly maps model pixel coordinates to on-screen positions under `object-fit: contain`.
- **Confidence scores** – Each detection shows class name + percentage (e.g. “motorcycle 81%).
- **Multi-object support** – Detects and lists every object returned by the model.
- **Responsive layout** – Works on different viewport sizes; image scales while boxes stay aligned.

3. Architecture

The application is built around three cooperating pieces:

1. Hidden HTML `<img>` element

Registered as a Flutter web platform view. It actually holds the picked image in the DOM and is the element passed to the TensorFlow model.

2. JavaScript interop layer

Functions `loadModel()` and `detectObjects(imgElementId)` live in `web/index.html`. They load coco-ssd once and return predictions as a JSON string.

3. Flutter UI (ObjectDetectionPage)

Picks the image, sets the `<img> src`, calls the JS function via `js_util.callMethod`, parses the result, and paints positioned bounding-box widgets on top of the platform view.

Data flow summary: **ImagePicker** → **bytes** → **Blob URL** → **ImageElement.src** → **onLoad** → **window.detectObjects()** → **JSON predictions** → **setState** → **Stack of Positioned boxes**.

4. Implementation Walkthrough

4.1 Platform View Registration

In `initState` the page creates an `html.ImageElement`, assigns it a fixed id (`detect-img`), and registers it once with `ui_web.platformViewRegistry.registerViewFactory`. A flag prevents double registration. The element is reused for every subsequent pick.

4.2 Pick → Detect Flow

- `ImagePicker.pickImage(source: ImageSource.gallery)` obtains an `XFile`.
- Bytes are wrapped in an `html.Blob` and turned into an object URL.
- The URL is assigned to the image element's `src`. The code awaits the `onLoad` event so that natural width/height become available (required later for coordinate mapping).
- Detection is invoked with:
`js_util.callMethod(html.window, 'detectObjects', [_imgElementId])`
- The returned JSON string is decoded into a list of maps containing `class`, `score` and `bbox` (`[x, y, width, height]`).

4.3 Bounding-Box Coordinate Mapping

Because the image is displayed with CSS `object-fit: contain`, the rendered size may be smaller than the widget and may be letter-boxed. The code computes the uniform scale factor and the centering offsets, then maps each model `bbox` from natural image pixels into Flutter widget coordinates:

```
scale = min(boxW / naturalW, boxH / naturalH)
offsetX = (boxW - renderedW) / 2    offsetY = (boxH - renderedH) / 2
x = offsetX + bbox[0] * scale      (same for y, width, height)
```

4.4 JavaScript Side (web/index.html)

Two CDN scripts load `TensorFlow.js` and the coco-ssd model. A small bootstrap script keeps a singleton model reference and exposes two global functions:

```
async function loadModel() { if (!cocoModel) cocoModel = await cocoSsd.load(); }

async function detectObjects(imgElementId) {
  if (!cocoModel) await loadModel();
  const imgEl = document.getElementById(imgElementId);
  const predictions = await cocoModel.detect(imgEl);
  return JSON.stringify(predictions);
}
```

5. Sample Results

The following screenshots were captured from the running Flutter Web application. Red bounding boxes and labels are drawn by Flutter widgets on top of the HTML image element.

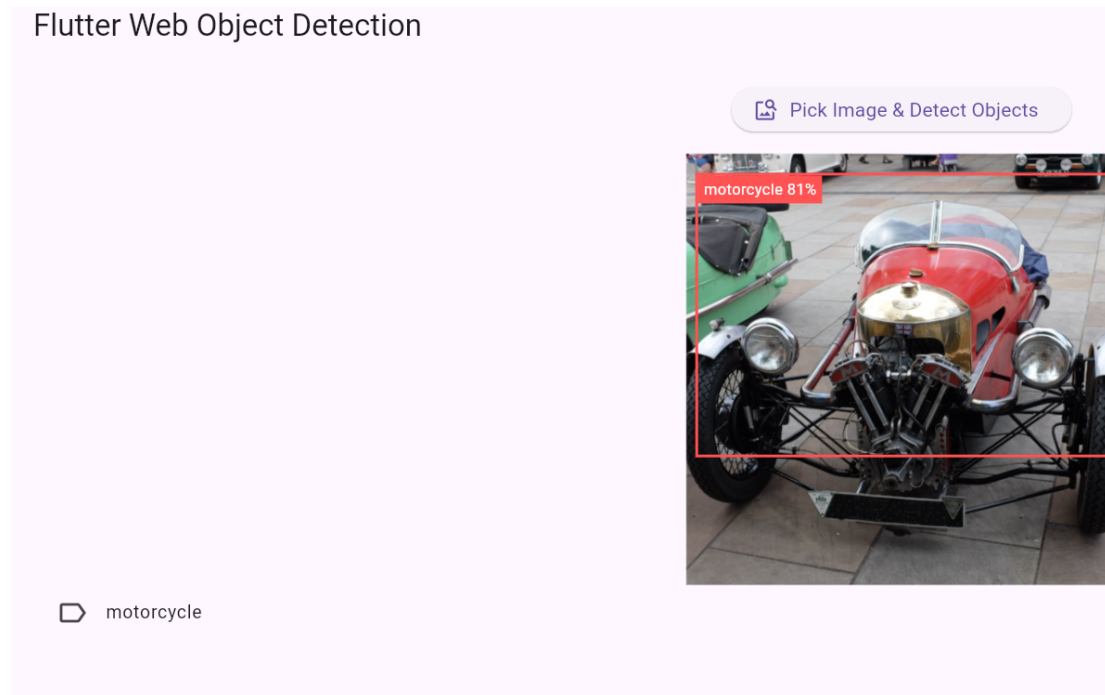


Figure 1 – Classic motorcycle / three-wheeler detected at 81% confidence.

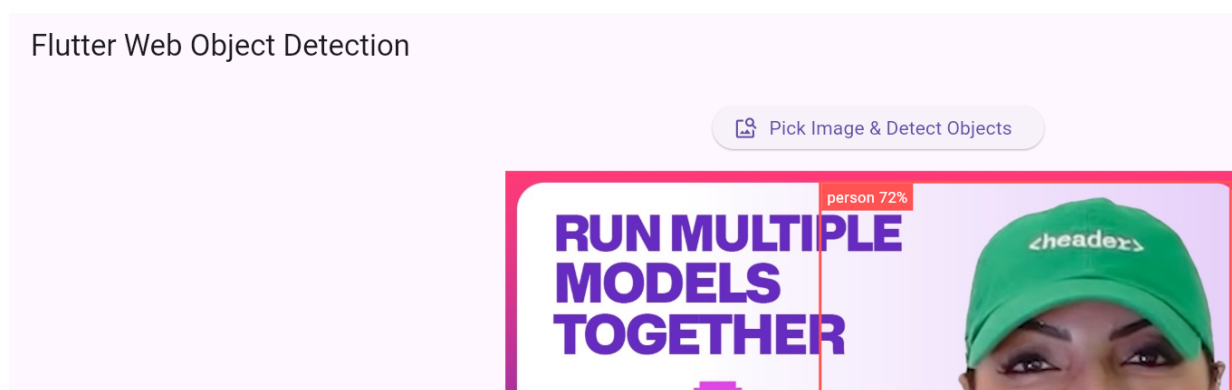


Figure 2 – Person detected at 72% confidence (promotional graphic).

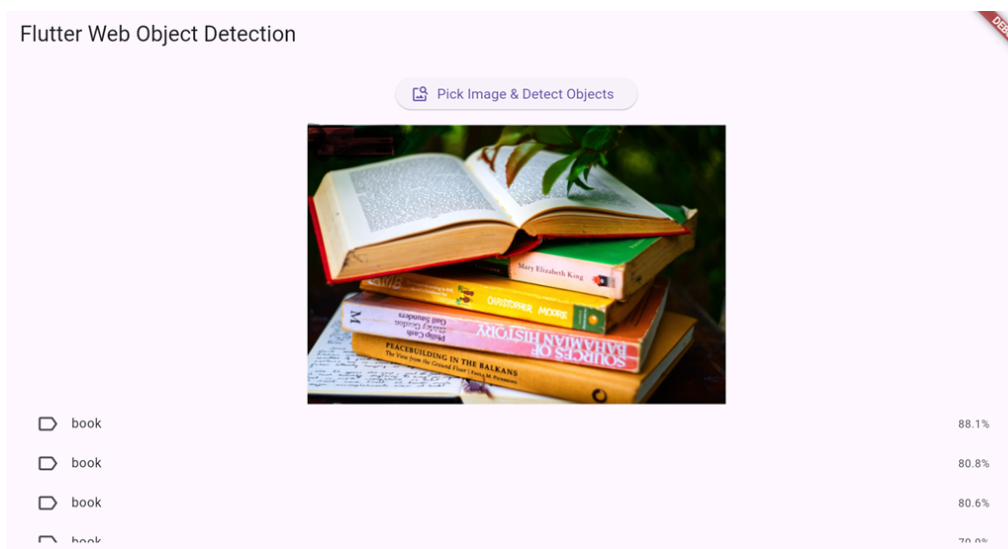


Figure 3 – Multiple books detected with confidences ranging from ~70% to 88%.

6. Dependencies

Package	Version / Source	Purpose
flutter	SDK	UI framework
image_picker	^1.1.2 / 1.2.3	Select image from gallery (web support)
js	^0.6.7	JS interop helpers (optional; code mainly uses dart:js_util)
@tensorflow/tfjs	4.20.0 (CDN)	TensorFlow.js core
@tensorflow-models/coco-ssd	2.2.3 (CDN)	Pretrained object-detection model

7. How to Run

1. Ensure Flutter SDK (≥ 3.0) with web support is installed.
2. Clone / open the project and run `flutter pub get`.
3. Launch with `flutter run -d chrome` (or any browser device).
4. Click “Pick Image & Detect Objects”, choose an image, and wait for the model to finish (first run downloads the coco-ssd weights).

Note: The app is intentionally web-only. The combination of `dart:html`, `dart:ui_web` and CDN scripts will not compile for mobile or desktop targets without conditional imports / alternative implementations.

8. Limitations & Possible Improvements

Current Limitations

- Web-only – relies on browser DOM and TensorFlow.js.
- First detection incurs model download latency (coco-ssd weights are fetched on demand).
- No camera / live video stream support yet (still-image only).
- Bounding-box styling is simple (solid red border + label); no color-per-class or opacity control.
- Default widget test is still the counter template and does not match the real app.

Suggested Improvements

- Pre-load the model at app startup (`window.loadModel()`) to hide the first-run delay.
- Add a full-screen loading overlay while inference runs.
- Revoke previous object URLs with `html.Url.revokeObjectUrl` to avoid memory leaks.
- Support live camera feed via `getUserMedia` + continuous detection loop.
- Color-code boxes by class and add a confidence threshold slider.
- Provide a mobile/desktop path using TFLite or a different inference backend.
- Write proper widget & integration tests for the detection page.

9. Conclusion

The project successfully demonstrates that Flutter Web can host a complete computer-vision pipeline by combining platform views, JavaScript interop, and a mature browser ML library. The coordinate-mapping logic correctly places bounding boxes under `object-fit: contain`, and the UI remains responsive and easy to understand. With the suggested improvements (model pre-loading, camera support, better testing) the same architecture can evolve into a more production-ready demo or be extended to other TensorFlow.js models.

— End of Report —